

Rapport final RLMinimax

Projet Pac-Man ALG4 - poids finaux, resultats et justification technique

Objectif du correctif

Le correctif final combine un agent Minimax avec elagage Alpha-Beta et une fonction d'evaluation affine. Les poids utilises par cette fonction sont appris par renforcement, puis stockes dans weights.json pour etre reutilises par RLMinimaxAgent.

Modele Fonction affine imposee par le sujet.	Recherche Distances BFS et exploration Alpha-Beta.	Poids 7 poids appris + 1 biais final.	Resultats Meilleur score moyen sur 10 layouts testes.
--	--	---	---

1. Conformite au sujet

Exigence	Implementation dans le projet	Statut
Facteurs Pac-Man	7 facteurs definis dans features.py : nourriture, fantomes, capsules, score.	OK
Recherche dans les facteurs	BFS utilise pour calculer les distances utiles dans le labyrinthe.	OK
Fonction affine	Evaluation sous la forme somme $a_i \cdot x_i(s) + C$.	OK
Minimax Alpha-Beta	RLMinimaxAgent explore les coups avec elagage Alpha-Beta.	OK
Poids appris	train.py ajuste les poids par apprentissage par renforcement.	OK
Comparaison	compare.py compare Minimax, AlphaBeta et RLMinimax.	OK
Statistiques	Scores moyens, winrate, temps moyens, coups moyens et Stop moyen.	OK

2. Fonction d'evaluation

Formule du sujet

$$f(s) = a_1x_1(s) + a_2x_2(s) + \dots + a_kx_k(s) + C$$

Fact.	Feature	Poids	Information	Role dans l'evaluation
x1	nearest_food_bfs	0.6891	Nourriture proche	Favorise la progression vers la nourriture.
x2	ghosts_within_3	-1.4093	Fantomes dangereux	Penalise les situations dangereuses.
x3	scared_ghosts_nearby	1.4061	Fantomes effrayes	Valorise les opportunités de points.
x4	remaining_food	0.4036	Nourriture restante	Encourage la fin de la carte.
x5	nearest_capsule_bfs	0.0252	Capsule proche	Ajoute un effet positif faible.
x6	current_score	1.8572	Score courant	Garde le lien avec le score officiel.
x7	nearest_dangerous_ghost_bfs	-1.2624	Danger principal	Penalise la proximite du danger.
C	bias	-0.0834	Constante	Correction globale independante de l'etat.

Interpretation des poids

- Les poids positifs valorisent les situations utiles : nourriture proche, fantomes effrayes et score courant.
- Les poids negatifs diminuent la valeur des etats proches de fantomes dangereux.
- Le biais C complete l'evaluation sans dependre directement de l'etat courant.

Resultats finaux

Comparaison relancee avec les poids actuels, sans nouvel entrainement

3. Scores obtenus

Layout	D	N	Minimax	AlphaBeta	RL	Diff	Win	Temps RL
testClassic	1	200	548.5	548.5	552.5	+4.0	100%	0.016s
smallClassic	2	100	-149.8	-149.8	196.6	+346.4	34%	0.284s
capsuleClassic	2	100	-228.9	-228.9	-135.2	+93.7	12%	0.304s
mediumClassic	1	50	14.5	14.5	515.5	+501.0	28%	0.219s
originalClassic	1	50	348.6	348.6	664.4	+315.7	2%	1.430s
trickyClassic	1	50	101.3	101.3	293.0	+191.7	2%	0.643s
contestClassic	1	50	11.3	11.3	244.4	+233.1	28%	0.214s
minimaxClassic	3	100	-243.5	-243.5	-153.1	+90.4	34%	0.036s
trappedClassic	1	1000	-392.3	-392.3	-391.4	+0.9	11%	0.001s
openClassic	1	10	-419.5	-745.4	1057.0	+1476.5	90%	0.521s

La colonne Diff est positive pour les 10 layouts testes : RLMinimax obtient donc un meilleur score moyen que Minimax dans cette serie de tests.

4. Gain moyen par layout

Gain moyen de RLMinimax par rapport a Minimax



5. Analyse

Lecture des resultats

- RLMinimax obtient le meilleur score moyen sur toutes les cartes testees.
- Le winrate varie selon la difficulte des cartes et le comportement des fantomes.
- Le Stop moyen vaut 0.0 : l'agent ne se bloque pas volontairement.
- Les resultats montrent que les poids appris ameliorent l'evaluation des positions.

Cas particulier

- openClassic est conserve dans les resultats car il montre aussi le temps de calcul.
- Les agents de reference depassent le timeout choisi sur cette carte.
- Ce resultat doit donc etre lu comme un cas particulier, pas comme le seul argument du projet.

Bilan des tests

Les tests confirment que la fonction affine apprise donne une evaluation plus efficace que les evaluations de reference utilisees dans ces comparaisons. Le projet respecte donc l'objectif principal : combiner Alpha-Beta avec une evaluation apprise par renforcement.

Contrôle technique

Fichiers principaux, commandes de vérification et conclusion du correctif

6. Fichiers principaux

Fichier	Rôle	Etat final
features.py	Calcul des facteurs $x_i(s)$.	7 facteurs disponibles.
rlMinimaxAgents.py	Agent final du projet.	Alpha-Beta + évaluation affine.
train.py	Apprentissage des poids.	Mise à jour des poids par renforcement.
compare.py	Comparaison expérimentale.	Minimax, AlphaBeta et RLMinimax testés.
weights.json	Poids finaux.	7 poids + 1 biais.
README.md	Présentation du projet.	Présent à la racine.

7. Vérifications effectuées

Vérification	Résultat
Compilation Python des fichiers principaux	OK
Correspondance entre 7 features et 7 poids	OK
Présence du biais C dans weights.json	OK
Comparaison sur 10 layouts	OK
Différence de score moyenne positive pour RLMinimax	OK
Stop moyen de RLMinimax	0.0 sur les tests relances

8. Limites

Limites du résultat

- Les poids finaux sont bons pour les tests effectués, mais ils ne prouvent pas une optimalité mathématique sur toutes les cartes possibles.
- Le score moyen est meilleur partout, mais le taux de victoire reste variable sur les cartes difficiles.
- Les fantômes gardent une part d'aléatoire, ce qui explique certaines variations entre les parties.

9. Conclusion

Conclusion finale

Le correctif final répond aux objectifs techniques du projet : l'agent RLMinimax utilise Alpha-Beta, une fonction d'évaluation affine, des facteurs liés à Pac-Man, des poids appris par renforcement et une comparaison statistique avec les agents de référence. Les résultats finaux montrent une amélioration du score moyen sur l'ensemble des layouts testés.

Fichiers à remettre

Le dossier final doit contenir le code, weights.json, README.md, les fichiers de résultats et ce rapport PDF. Les poids sont placés à la racine du projet afin d'être visibles directement.